JAVA DYNAMIC PROGRAMMING

Complete Concepts • Real IntelliJ Project Structure • Runnable Java Code • Interview Patterns

A practical interview-preparation handbook. Every code section contains actual Java source—not folder placeholders or pseudocode.

Language: Java | IDE: IntelliJ IDEA | Focus: Product-based & high-tech interviews

How to use this guide

This guide is organized as a real Java project. Each algorithm is shown with its package, exact file name, imports, class, and runnable main method. Create the matching package in IntelliJ and paste the code into the named .java file. The class name must match the file name.

Project structure

```
DynamicProgramming/
■■■ src/
    ■■■ dp/
        ■■■ basics/
        ■   ■■■ FibonacciMemo.java
        ■   ■■■ FibonacciTabulation.java
        ■   ■■■ ClimbingStairs.java
        ■   ■■■ MinCostClimbingStairs.java
        ■   ■■■ HouseRobber.java
        ■   ■■■ HouseRobberCircular.java
        ■■■ grid/
        ■   ■■■ UniquePaths.java
        ■   ■■■ UniquePathsWithObstacles.java
        ■   ■■■ MinimumPathSum.java
        ■   ■■■ MaximalSquare.java
        ■■■ knapsack/
        ■   ■■■ ZeroOneKnapsack.java
        ■   ■■■ UnboundedKnapsack.java
        ■   ■■■ SubsetSum.java
        ■   ■■■ EqualPartition.java
        ■   ■■■ CountSubsets.java
        ■   ■■■ MinCoins.java
        ■   ■■■ CoinChangeWays.java
        ■   ■■■ TargetSum.java
        ■■■ sequence/
        ■   ■■■ LIS.java
        ■   ■■■ LongestCommonSubstring.java
        ■   ■■■ LCS.java
        ■   ■■■ EditDistance.java
        ■   ■■■ LongestPalindromicSubsequence.java
        ■■■ interval/
        ■   ■■■ MatrixChainMultiplication.java
        ■   ■■■ BurstBalloons.java
        ■   ■■■ PalindromePartitioning.java
        ■■■ stocks/
        ■   ■■■ StockOneTransaction.java
        ■   ■■■ StockUnlimited.java
        ■   ■■■ StockCooldown.java
        ■   ■■■ StockKTransactions.java
        ■■■ trees/
        ■   ■■■ TreeMaximumIndependentSet.java
        ■■■ bitmask/
        ■   ■■■ TravellingSalesperson.java
        ■■■ digit/
        ■   ■■■ DigitDPCount.java
        ■■■ advanced/
            ■■■ LISCount.java
```

IntelliJ: create a Java project, create packages such as dp.basics, then create each public class in its own same-named file. Run that class's main method. Every code block below includes its package and import statements.

1. DP fundamentals: the complete problem-solving method

What Dynamic Programming means

Dynamic Programming (DP) solves a problem by defining smaller subproblems, solving each relevant subproblem once, and reusing its answer. DP is useful when the problem has overlapping subproblems and optimal substructure.

The 7-step DP design checklist

• State: define exactly what dp[index] or dp[index][state] means. Say it in one sentence.

• Choices: list every legal decision available from the current state.

• Transition: express the current answer using smaller states.

• Base cases: define the smallest states directly; do not leave them implicit.

• Evaluation order: choose recursion + memoization or bottom-up tabulation in dependency order.

• Answer location: identify whether the answer is dp[n], dp[n-1], max over a row, or a separate accumulated value.

• Complexity: count the number of distinct states × work per state; also account for memory and recursion stack.

Memoization vs tabulation

Memoization: top-down recursion plus a cache. Good when only a fraction of states is visited; watch recursion depth. Tabulation: bottom-up loops. Usually avoids recursion overhead and makes space optimization easier.

Common DP categories

• 1D / linear DP: Fibonacci, stairs, robber, decode ways.

• Grid DP: unique paths, obstacles, minimum path, falling path.

• Knapsack / subset DP: choose or skip items, capacity or target sum.

• String / sequence DP: LCS, edit distance, LIS, palindromic subsequence.

• Interval DP: split an interval or choose the last operation.

• State-machine DP: stock trading, cooldown, transaction limits.

• Tree DP: combine answers from child subtrees.

• Bitmask DP: represent a selected subset using bits.

• Digit DP: count numbers satisfying digit constraints.

• DP optimization: monotonic queue, convex hull trick, divide-and-conquer optimization, Knuth optimization.

Loop-direction rules

• 0/1 knapsack: capacity loops downward so one item is not reused in the same iteration.

• Unbounded knapsack / unlimited coin use: capacity loops upward so the current item can be reused.

• Counting combinations: process coin/item outside, sum inside.

• Counting ordered sequences: sum/target outside, choices inside.

• Interval DP: process shorter intervals before longer intervals.

• Bitmask DP: transition from a smaller mask to a mask with one extra bit.

Java safety checklist

• Use long when sums/products may exceed int; Java int max is 2,147,483,647.

• Use Arrays.fill for sentinel initialization; avoid confusing 0 with uncomputed when 0 is a valid answer.

• Guard unreachable states before adding costs to INF.

• Do not use recursion for extremely deep input unless stack depth is safe.

• Check empty arrays, one element, zero target, duplicate values, and impossible states.

Fibonacci — memoization

File: src/dp/basics/Fibonacci.java

Package: dp.basics | State / implementation rule: State: fib(n) = nth Fibonacci number. Transition: fib(n)=fib(n-1)+fib(n-2); bases 0 and 1. Cache prevents repeated recursion.

```
package dp.basics;
import java.util.Arrays;

public class FibonacciMemo {
    static long fib(int n, long[] memo) {
        if (n <= 1) return n;
        if (memo[n] != -1) return memo[n];
        return memo[n] = fib(n - 1, memo) + fib(n - 2, memo);
    }
```

```
    public static void main(String[] args) {
        int n = 10;
        long[] memo = new long[n + 1];
        Arrays.fill(memo, -1);
        System.out.println(fib(n, memo)); // 55
    }
}
```

Complexity: Time O(n), space O(n) cache + O(n) recursion stack.

Fibonacci — tabulation

File: src/dp/basics/Fibonacci.java

Package: dp.basics | State / implementation rule: Build answers from 0 upward because each state depends on the previous two. Only two previous values are needed, so the array can be compressed to two variables.

```
package dp.basics;
public class FibonacciTabulation {
    static long fib(int n) {
        if (n <= 1) return n;
        long[] dp = new long[n + 1];
        dp[0] = 0; dp[1] = 1;
        for (int i = 2; i <= n; i++)
            dp[i] = dp[i - 1] + dp[i - 2];
        return dp[n];
    }
    public static void main(String[] args) {
        System.out.println(fib(10)); // 55
    }
}
```

Complexity: Time O(n), space O(n); optimized space O(1).

Climbing Stairs

File: src/dp/basics/ClimbingStairs.java

Package: dp.basics | State / implementation rule: State: ways[i] = ways to reach stair i. Last move is 1 or 2 steps: ways[i]=ways[i-1]+ways[i-2].

```
package dp.basics;
public class ClimbingStairs {
    static int climb(int n) {
        if (n <= 2) return n;
        int twoBack = 1, oneBack = 2;
        for (int step = 3; step <= n; step++) {
            int current = oneBack + twoBack;
            twoBack = oneBack;
            oneBack = current;
        }
        return oneBack;
    }
    public static void main(String[] args) {
        System.out.println(climb(5)); // 8
    }
}
```

Complexity: Time O(n), space O(1).

Minimum Cost Climbing Stairs

File: src/dp/basics/MinimumCostClimbingStairs.java

Package: dp.basics | State / implementation rule: You may start at index 0 or 1 and pay the cost of the step you land on. Compute the cheapest cost to reach the top beyond the last index.

```
package dp.basics;
public class MinCostClimbingStairs {
    static int minCost(int[] cost) {
        int prev2 = 0, prev1 = 0;
        for (int i = 2; i <= cost.length; i++) {
            int current = Math.min(prev1 + cost[i - 1],
                                   prev2 + cost[i - 2]);
            prev2 = prev1;
            prev1 = current;
        }
        return prev1;
    }
    public static void main(String[] args) {
        System.out.println(minCost(new int[]{10, 15, 20})); // 15
```

```
    }
}
```

Complexity: Time O(n), space O(1).

House Robber

File: src/dp/basics/HouseRobber.java

Package: dp.basics | State / implementation rule: At each house choose skip or rob. dp[i]=max(dp[i-1], nums[i]+dp[i-2]). Adjacent houses cannot both be selected.

```
package dp.basics;
public class HouseRobber {
    static int rob(int[] nums) {
        int skipPrevious = 0, takeOrSkipPrevious = 0;
        for (int value : nums) {
            int current = Math.max(takeOrSkipPrevious,
                                   skipPrevious + value);
            skipPrevious = takeOrSkipPrevious;
            takeOrSkipPrevious = current;
        }
        return takeOrSkipPrevious;
    }
    public static void main(String[] args) {
        System.out.println(rob(new int[]{2, 7, 9, 3, 1})); // 12
    }
}
```

Complexity: Time O(n), space O(1).

House Robber II — circular houses

File: src/dp/basics/HouseRobberII.java

Package: dp.basics | State / implementation rule: First and last houses are adjacent. Therefore either exclude the last house and solve [0,n-2], or exclude the first and solve [1,n-1].

```
package dp.basics;
public class HouseRobberCircular {
    static int linear(int[] a, int left, int right) {
        int prev2 = 0, prev1 = 0;
        for (int i = left; i <= right; i++) {
            int cur = Math.max(prev1, prev2 + a[i]);
            prev2 = prev1; prev1 = cur;
        }
        return prev1;
    }
    static int rob(int[] nums) {
        int n = nums.length;
        if (n == 0) return 0;
        if (n == 1) return nums[0];
        return Math.max(linear(nums, 0, n - 2),
                        linear(nums, 1, n - 1));
    }
    public static void main(String[] args) {
        System.out.println(rob(new int[]{2, 3, 2})); // 3
    }
}
```

Complexity: Time O(n), space O(1).

Unique Paths

File: src/dp/grid/UniquePaths.java

Package: dp.grid | State / implementation rule: From each cell, arrive from above or left: dp[r][c]=dp[r-1][c]+dp[r][c-1]. First row and column contain one path. One row is enough if columns are updated left-to-right.

```
package dp.grid;
public class UniquePaths {
    static int paths(int m, int n) {
        int[] dp = new int[n];
        java.util.Arrays.fill(dp, 1);
        for (int r = 1; r < m; r++)
            for (int c = 1; c < n; c++)
                dp[c] += dp[c - 1];
        return dp[n - 1];
    }
```

```
    public static void main(String[] args) {
        System.out.println(paths(3, 7)); // 28
    }
}
```

Complexity: Time O(mn), space O(n).

Unique Paths with Obstacles

File: src/dp/grid/UniquePathswithObstacles.java

Package: dp.grid | State / implementation rule: An obstacle resets the number of ways to zero. dp[c] before update is the value from above; dp[c-1] is the updated value from the left.

```
package dp.grid;
public class UniquePathsWithObstacles {
    static int paths(int[][] grid) {
        int m = grid.length, n = grid[0].length;
        long[] dp = new long[n];
        dp[0] = grid[0][0] == 1 ? 0 : 1;
        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                if (grid[r][c] == 1) dp[c] = 0;
                else if (c > 0) dp[c] += dp[c - 1];
            }
        }
        return (int) dp[n - 1];
    }
    public static void main(String[] args) {
        int[][] g = {{0,0,0},{0,1,0},{0,0,0}};
        System.out.println(paths(g)); // 2
    }
}
```

Complexity: Time O(mn), space O(n).

Minimum Path Sum

File: src/dp/grid/MinimumPathSum.java

Package: dp.grid | State / implementation rule: Minimum instead of sum of ways: dp[r][c]=grid[r][c]+min(top,left).

```
package dp.grid;
public class MinimumPathSum {
    static int minPath(int[][] a) {
        int m = a.length, n = a[0].length;
        int[] dp = new int[n];
        dp[0] = a[0][0];
        for (int c = 1; c < n; c++) dp[c] = dp[c-1] + a[0][c];
        for (int r = 1; r < m; r++) {
            dp[0] += a[r][0];
            for (int c = 1; c < n; c++)
                dp[c] = a[r][c] + Math.min(dp[c], dp[c-1]);
        }
        return dp[n-1];
    }
    public static void main(String[] args) {
        int[][] a = {{1,3,1},{1,5,1},{4,2,1}};
        System.out.println(minPath(a)); // 7
    }
}
```

Complexity: Time O(mn), space O(n).

0/1 Knapsack

File: src/dp/knapsack/0/1Knapsack.java

Package: dp.knapsack | State / implementation rule: For each item: take it once or skip it. Descending capacity is essential; ascending would allow reusing the same item. dp[c] means maximum value with capacity c.

```
package dp.knapsack;
public class ZeroOneKnapsack {
    static int solve(int[] wt, int[] val, int capacity) {
        int[] dp = new int[capacity + 1];
        for (int i = 0; i < wt.length; i++) {
            for (int c = capacity; c >= wt[i]; c--) {
                dp[c] = Math.max(dp[c], val[i] + dp[c - wt[i]]);
            }
        }
```

```
        return dp[capacity];
    }
    public static void main(String[] args) {
        int[] wt = {1, 3, 4, 5}, val = {1, 4, 5, 7};
        System.out.println(solve(wt, val, 7)); // 9
    }
}
```

Complexity: Time O(nW), space O(W).

Unbounded Knapsack

File: src/dp/knapsack/UnboundedKnapsack.java

Package: dp.knapsack | State / implementation rule: Items may be reused. Ascending capacity lets dp[c-wt[i]] already include the current item.

```
package dp.knapsack;
public class UnboundedKnapsack {
    static int solve(int[] wt, int[] val, int capacity) {
        int[] dp = new int[capacity + 1];
        for (int i = 0; i < wt.length; i++) {
            for (int c = wt[i]; c <= capacity; c++) {
                dp[c] = Math.max(dp[c], val[i] + dp[c - wt[i]]);
            }
        }
        return dp[capacity];
    }
    public static void main(String[] args) {
        int[] wt = {2, 3, 4}, val = {4, 5, 7};
        System.out.println(solve(wt, val, 7)); // 13
    }
}
```

Complexity: Time O(nW), space O(W).

Subset Sum

File: src/dp/knapsack/SubsetSum.java

Package: dp.knapsack | State / implementation rule: dp[s] means some subset of processed elements totals s. Descending sum prevents using an element more than once.

```
package dp.knapsack;
public class SubsetSum {
    static boolean canMake(int[] a, int target) {
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;
        for (int x : a)
            for (int s = target; s >= x; s--)
                dp[s] |= dp[s - x];
        return dp[target];
    }
    public static void main(String[] args) {
        System.out.println(canMake(new int[]{3, 34, 4, 12, 5, 2}, 9)); // true
    }
}
```

Complexity: Time O(nT), space O(T).

Equal Subset Partition

File: src/dp/knapsack/EqualSubsetPartition.java

Package: dp.knapsack | State / implementation rule: Equal partition exists exactly when total sum is even and a subset totals sum/2. Assumes non-negative values.

```
package dp.knapsack;
public class EqualPartition {
    static boolean canPartition(int[] nums) {
        int sum = 0;
        for (int x : nums) sum += x;
        if (sum % 2 != 0) return false;
        int target = sum / 2;
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;
        for (int x : nums)
            for (int s = target; s >= x; s--)
                dp[s] |= dp[s - x];
        return dp[target];
    }
```

```
    public static void main(String[] args) {
        System.out.println(canPartition(new int[]{1,5,11,5})); // true
    }
}
```

Complexity: Time O(nS), space O(S), S=total sum/2.

Count Subsets with Given Sum

File: src/dp/knapsack/CountSubsetswithGivenSum.java

Package: dp.knapsack | State / implementation rule: Same subset transition, but add counts instead of OR. dp[0]=1 represents the empty subset. Zero-valued elements double counts, so the loop correctly handles them.

```
package dp.knapsack;
public class CountSubsets {
    static long count(int[] a, int target) {
        long[] dp = new long[target + 1];
        dp[0] = 1;
        for (int x : a)
            for (int s = target; s >= x; s--)
                dp[s] += dp[s - x];
        return dp[target];
    }
    public static void main(String[] args) {
        System.out.println(count(new int[]{1,2,3,3}, 6)); // 3
    }
}
```

Complexity: Time O(nT), space O(T).

Minimum Coins

File: src/dp/knapsack/MinimumCoins.java

Package: dp.knapsack | State / implementation rule: Unbounded minimum-count DP. dp[x]=minimum coins for amount x. If unreachable, return -1. Coin loop outside is also valid for minimum count.

```
package dp.knapsack;
import java.util.Arrays;
public class MinCoins {
    static int minCoins(int[] coins, int amount) {
        int inf = amount + 1;
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, inf);
        dp[0] = 0;
        for (int coin : coins)
            for (int x = coin; x <= amount; x++)
                if (dp[x - coin] != inf)
                    dp[x] = Math.min(dp[x], dp[x - coin] + 1);
        return dp[amount] == inf ? -1 : dp[amount];
    }
    public static void main(String[] args) {
        System.out.println(minCoins(new int[]{1,2,5}, 11)); // 3
    }
}
```

Complexity: Time O(CA), space O(A).

Coin Change — number of combinations

File: src/dp/knapsack/CoinChange.java

Package: dp.knapsack | State / implementation rule: Coin outer loop counts combinations where order does not matter. If amount were the outer loop, permutations/orderings would be counted.

```
package dp.knapsack;
public class CoinChangeWays {
    static long ways(int[] coins, int amount) {
        long[] dp = new long[amount + 1];
        dp[0] = 1;
        for (int coin : coins)
            for (int x = coin; x <= amount; x++)
                dp[x] += dp[x - coin];
        return dp[amount];
    }
    public static void main(String[] args) {
        System.out.println(ways(new int[]{1,2,5}, 5)); // 4
    }
}
```

Complexity: Time O(CA), space O(A).

Target Sum

File: src/dp/knapsack/TargetSum.java

Package: dp.knapsack | State / implementation rule: Transform plus/minus assignment into subset count: let P-N=target and P+N=sum, so P=(sum+target)/2. Then count subsets with that sum. Assumes non-negative nums.

```
package dp.knapsack;
public class TargetSum {
    static int findWays(int[] nums, int target) {
        int sum = 0;
        for (int x : nums) sum += x;
        if (Math.abs(target) > sum || (sum + target) % 2 != 0) return 0;
        int wanted = (sum + target) / 2;
        long[] dp = new long[wanted + 1];
        dp[0] = 1;
        for (int x : nums)
            for (int s = wanted; s >= x; s--)
                dp[s] += dp[s - x];
        return (int) dp[wanted];
    }
    public static void main(String[] args) {
        System.out.println(findWays(new int[]{1,1,1,1,1}, 3)); // 5
    }
}
```

Complexity: Time O(nS), space O(S).

Longest Increasing Subsequence — O(n log n)

File: src/dp/sequence/LongestIncreasingSubsequence.java

Package: dp.sequence | State / implementation rule: tails[k] is the smallest possible ending value of an increasing subsequence of length k+1. Lower bound (first >= x) gives strictly increasing LIS; use upper bound (first > x) for non-decreasing subsequence.

```
package dp.sequence;
import java.util.Arrays;
public class LIS {
    static int lengthOfLIS(int[] nums) {
        int[] tails = new int[nums.length];
        int size = 0;
        for (int x : nums) {
            int left = 0, right = size;
            while (left < right) {
                int mid = left + (right - left) / 2;
                if (tails[mid] < x) left = mid + 1;
                else right = mid;
            }
            tails[left] = x;
            if (left == size) size++;
        }
        return size;
    }
    public static void main(String[] args) {
        System.out.println(lengthOfLIS(new int[]{10,9,2,5,3,7,101,18})); // 4
    }
}
```

Complexity: Time O(n log n), space O(n).

Longest Common Subsequence

File: src/dp/sequence/LongestCommonSubsequence.java

Package: dp.sequence | State / implementation rule: If last characters match: 1+diagonal. Otherwise discard one character from either string and take max. The variable diagonal preserves the old dp[i-1][j-1].

```
package dp.sequence;
public class LCS {
    static int lcs(String a, String b) {
        int[] dp = new int[b.length() + 1];
        for (int i = 1; i <= a.length(); i++) {
            int diagonal = 0;
            for (int j = 1; j <= b.length(); j++) {
                int oldAbove = dp[j];
                if (a.charAt(i-1) == b.charAt(j-1))
```

```
                    dp[j] = diagonal + 1;
                else
                    dp[j] = Math.max(dp[j], dp[j-1]);
                diagonal = oldAbove;
            }
        }
        return dp[b.length()];
    }
    public static void main(String[] args) {
        System.out.println(lcs("abcde", "ace")); // 3
    }
}
```

Complexity: Time O(mn), space O(n).

Longest Common Substring

File: src/dp/sequence/LongestCommonSubstring.java

Package: dp.sequence | State / implementation rule: Substring must be contiguous, so a mismatch resets the state to zero. Iterate j backward to preserve the previous row's diagonal.

```
package dp.sequence;
public class LongestCommonSubstring {
    static int longest(String a, String b) {
        int[] dp = new int[b.length() + 1];
        int best = 0;
        for (int i = 1; i <= a.length(); i++) {
            for (int j = b.length(); j >= 1; j--) {
                if (a.charAt(i-1) == b.charAt(j-1))
                    dp[j] = dp[j-1] + 1;
                else
                    dp[j] = 0;
                best = Math.max(best, dp[j]);
            }
        }
        return best;
    }
    public static void main(String[] args) {
        System.out.println(longest("abcdxyz", "xyzabcd")); // 4
    }
}
```

Complexity: Time O(mn), space O(n).

Edit Distance

File: src/dp/sequence/EditDistance.java

Package: dp.sequence | State / implementation rule: Operations: insert, delete, replace. If chars match, no operation; otherwise 1+min(insert, delete, replace). dp[i][j] converts first i chars to first j chars.

```
package dp.sequence;
public class EditDistance {
    static int distance(String a, String b) {
        int[] dp = new int[b.length() + 1];
        for (int j = 0; j <= b.length(); j++) dp[j] = j;
        for (int i = 1; i <= a.length(); i++) {
            int diagonal = dp[0];
            dp[0] = i;
            for (int j = 1; j <= b.length(); j++) {
                int above = dp[j];
                if (a.charAt(i-1) == b.charAt(j-1)) dp[j] = diagonal;
                else dp[j] = 1 + Math.min(diagonal,
                                 Math.min(dp[j], dp[j-1]));
                diagonal = above;
            }
        }
        return dp[b.length()];
    }
    public static void main(String[] args) {
        System.out.println(distance("horse", "ros")); // 3
    }
}
```

Complexity: Time O(mn), space O(n).

Longest Palindromic Subsequence

File: src/dp/sequence/LongestPalindromicSubsequence.java

Package: dp.sequence | State / implementation rule: LPS(s)=LCS(s, reverse(s)). This computes the length; reconstructing the actual subsequence requires storing parent/choice information or a full table.

```
package dp.sequence;
public class LongestPalindromicSubsequence {
    static int solve(String s) {
        String r = new StringBuilder(s).reverse().toString();
        int[] dp = new int[r.length() + 1];
        for (int i = 1; i <= s.length(); i++) {
            int diagonal = 0;
            for (int j = 1; j <= r.length(); j++) {
                int above = dp[j];
                if (s.charAt(i-1) == r.charAt(j-1)) dp[j] = diagonal + 1;
                else dp[j] = Math.max(dp[j], dp[j-1]);
                diagonal = above;
            }
        }
        return dp[r.length()];
    }
    public static void main(String[] args) {
        System.out.println(solve("bbbab")); // 4
    }
}
```

Complexity: Time O(n²), space O(n).

Matrix Chain Multiplication

File: src/dp/interval/MatrixChainMultiplication.java

Package: dp.interval | State / implementation rule: dp[i][j] is minimum scalar multiplications for matrices i..j. Try every final split k. The multiplication cost is dimensions[i] × dimensions[k+1] × dimensions[j+1].

```
package dp.interval;
public class MatrixChainMultiplication {
    static long minCost(int[] p) {
        int n = p.length - 1;
        long[][] dp = new long[n][n];
        for (int len = 2; len <= n; len++) {
            for (int i = 0; i + len - 1 < n; i++) {
                int j = i + len - 1;
                dp[i][j] = Long.MAX_VALUE;
                for (int k = i; k < j; k++) {
                    long cost = dp[i][k] + dp[k+1][j]
                              + 1L * p[i] * p[k+1] * p[j+1];
                    dp[i][j] = Math.min(dp[i][j], cost);
                }
            }
        }
        return n == 0 ? 0 : dp[0][n-1];
    }
    public static void main(String[] args) {
        System.out.println(minCost(new int[]{40,20,30,10,30})); // 26000
    }
}
```

Complexity: Time O(n³), space O(n²).

Burst Balloons

File: src/dp/interval/BurstBalloons.java

Package: dp.interval | State / implementation rule: Choose the last balloon to burst in interval [left,right]. Its neighbors are then the fixed boundaries left-1 and right+1. This reverse-choice view is the key interval-DP trick.

```
package dp.interval;
public class BurstBalloons {
    static int maxCoins(int[] nums) {
        int n = nums.length;
        int[] a = new int[n + 2];
        a[0] = a[n + 1] = 1;
        System.arraycopy(nums, 0, a, 1, n);
        int[][] dp = new int[n + 2][n + 2];
        for (int len = 1; len <= n; len++) {
            for (int left = 1; left + len - 1 <= n; left++) {
                int right = left + len - 1;
                for (int last = left; last <= right; last++) {
                    int coins = a[left-1] * a[last] * a[right+1]
                              + dp[left][last-1] + dp[last+1][right];
                    dp[left][right] = Math.max(dp[left][right], coins);
                }
            }
        }
        return dp[1][n];
```

```
    }
    public static void main(String[] args) {
        System.out.println(maxCoins(new int[]{3,1,5,8})); // 167
    }
}
```

Complexity: Time O(n³), space O(n²).

Palindrome Partitioning II — minimum cuts

File: src/dp/interval/PalindromePartitioningII.java

Package: dp.interval | State / implementation rule: Precompute palindromic intervals while scanning i backward. cuts[i] is minimum cuts needed for suffix starting at i. A palindrome [i,j] allows 1+cuts[j+1].

```
package dp.interval;
public class PalindromePartitioning {
    static int minCuts(String s) {
        int n = s.length();
        boolean[][] pal = new boolean[n][n];
        int[] cuts = new int[n + 1];
        for (int i = n - 1; i >= 0; i--) {
            cuts[i] = n - i - 1;
            for (int j = i; j < n; j++) {
                if (s.charAt(i) == s.charAt(j) &&
                    (j - i < 2 || pal[i+1][j-1])) {
                    pal[i][j] = true;
                    cuts[i] = Math.min(cuts[i], 1 + cuts[j+1]);
                }
            }
        }
        return n == 0 ? 0 : cuts[0];
    }
    public static void main(String[] args) {
        System.out.println(minCuts("aab")); // 1
    }
}
```

Complexity: Time O(n²), space O(n²).

Stock — at most one transaction

File: src/dp/stocks/Stock.java

Package: dp.stocks | State / implementation rule: Track the minimum price seen before/at today and the best sell profit. Equivalent to a two-state DP, compressed to two variables.

```
package dp.stocks;
public class StockOneTransaction {
    static int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE, best = 0;
        for (int p : prices) {
            minPrice = Math.min(minPrice, p);
            best = Math.max(best, p - minPrice);
        }
        return best;
    }
    public static void main(String[] args) {
        System.out.println(maxProfit(new int[]{7,1,5,3,6,4})); // 5
    }
}
```

Complexity: Time O(n), space O(1).

Stock — unlimited transactions

File: src/dp/stocks/Stock.java

Package: dp.stocks | State / implementation rule: With unlimited transactions and no fee/cooldown, collect every positive adjacent rise. This equals optimal buy/sell segmentation.

```
package dp.stocks;
public class StockUnlimited {
    static int maxProfit(int[] prices) {
        int profit = 0;
        for (int i = 1; i < prices.length; i++)
            if (prices[i] > prices[i-1])
                profit += prices[i] - prices[i-1];
        return profit;
    }
    public static void main(String[] args) {
```

```
        System.out.println(maxProfit(new int[]{7,1,5,3,6,4})); // 7
    }
}
```

Complexity: Time O(n), space O(1).

Stock with cooldown

File: src/dp/stocks/Stockwithcooldown.java

Package: dp.stocks | State / implementation rule: States: hold (own stock), sold (sold today), rest (not holding and not just sold). Buy only from rest, which enforces one cooldown day after selling.

```
package dp.stocks;
public class StockCooldown {
    static int maxProfit(int[] p) {
        int hold = -p[0], sold = 0, rest = 0;
        for (int i = 1; i < p.length; i++) {
            int oldHold = hold, oldSold = sold, oldRest = rest;
            hold = Math.max(oldHold, oldRest - p[i]);
            sold = oldHold + p[i];
            rest = Math.max(oldRest, oldSold);
        }
        return Math.max(sold, rest);
    }
    public static void main(String[] args) {
        System.out.println(maxProfit(new int[]{1,2,3,0,2})); // 3
    }
}
```

Complexity: Time O(n), space O(1).

Stock with at most k transactions

File: src/dp/stocks/Stockwithatmostktransactions.java

Package: dp.stocks | State / implementation rule: buy[t] = best profit after buying for transaction t; sell[t] = best profit after completing t sales. When k is large enough, the problem reduces to unlimited transactions.

```
package dp.stocks;
public class StockKTransactions {
    static int maxProfit(int k, int[] prices) {
        int n = prices.length;
        if (n < 2 || k == 0) return 0;
        if (k >= n / 2) {
            int ans = 0;
            for (int i = 1; i < n; i++)
                ans += Math.max(0, prices[i] - prices[i-1]);
            return ans;
        }
        int[] buy = new int[k + 1];
        int[] sell = new int[k + 1];
        java.util.Arrays.fill(buy, Integer.MIN_VALUE / 2);
        for (int price : prices) {
            for (int t = 1; t <= k; t++) {
                buy[t] = Math.max(buy[t], sell[t-1] - price);
                sell[t] = Math.max(sell[t], buy[t] + price);
            }
        }
        return sell[k];
    }
    public static void main(String[] args) {
        System.out.println(maxProfit(2, new int[]{3,2,6,5,0,3})); // 7
    }
}
```

Complexity: Time O(nk), space O(k).

Tree DP — maximum weight independent set

File: src/dp/trees/TreeDP.java

Package: dp.trees | State / implementation rule: Return two states per node: index 0 = take this node, index 1 = skip this node. If take, children must be skipped. If skip, each child may be taken or skipped. This example uses a binary tree.

```
package dp.trees;
public class TreeMaximumIndependentSet {
    static class Node {
        int value; Node left, right;
```

```
        Node(int value) { this.value = value; }
    }
    static int[] solve(Node node) {
        if (node == null) return new int[]{0, 0};
        int[] l = solve(node.left), r = solve(node.right);
        int take = node.value + l[1] + r[1];
        int skip = Math.max(l[0], l[1]) + Math.max(r[0], r[1]);
        return new int[]{take, skip};
    }
    public static void main(String[] args) {
        Node root = new Node(10);
        root.left = new Node(1); root.right = new Node(2);
        root.left.left = new Node(10); root.right.right = new Node(10);
        int[] ans = solve(root);
        System.out.println(Math.max(ans[0], ans[1])); // 22
    }
}
```

Complexity: Time O(n), space O(h) recursion stack.

Bitmask DP — Travelling Salesperson

File: src/dp/bitmask/BitmaskDP.java

Package: dp.bitmask | State / implementation rule: State (mask,u): visited cities are encoded in mask and current city is u. Try every unvisited next city. This exact implementation assumes a complete graph and n small enough for 1<<n and memory.

```
package dp.bitmask;
import java.util.Arrays;
public class TravellingSalesperson {
    static int tsp(int[][] cost) {
        int n = cost.length, full = (1 << n) - 1;
        int[][] memo = new int[1 << n][n];
        for (int[] row : memo) Arrays.fill(row, -1);
        return dfs(1, 0, full, cost, memo);
    }
    static int dfs(int mask, int u, int full, int[][] cost, int[][] memo) {
        if (mask == full) return cost[u][0];
        if (memo[mask][u] != -1) return memo[mask][u];
        int best = Integer.MAX_VALUE / 2;
        for (int v = 0; v < cost.length; v++) {
            if ((mask & (1 << v)) == 0) {
                best = Math.min(best, cost[u][v] +
                    dfs(mask | (1 << v), v, full, cost, memo));
            }
        }
        return memo[mask][u] = best;
    }
    public static void main(String[] args) {
        int[][] c = {{0,10,15,20},{10,0,35,25},{15,35,0,30},{20,25,30,0}};
        System.out.println(tsp(c)); // 80
    }
}
```

Complexity: Time O(n²·2^n), space O(n·2^n).

Digit DP — count numbers from 0 to N with digit sum <= K

File: src/dp/digit/DigitDP.java

Package: dp.digit | State / implementation rule: State: position, digit sum so far, tight flag. tight=true means the prefix equals N's prefix, so the next digit cannot exceed N's current digit. Leading zeroes naturally represent shorter numbers. This version assumes N>=0 and modest K.

```
package dp.digit;
import java.util.Arrays;
public class DigitDPCount {
    static String bound;
    static int limit;
    static long[][][] memo;
    static long dfs(int pos, int sum, boolean tight) {
        if (sum > limit) return 0;
        if (pos == bound.length()) return 1;
        int t = tight ? 1 : 0;
        if (memo[pos][sum][t] != -1) return memo[pos][sum][t];
        int maxDigit = tight ? bound.charAt(pos) - '0' : 9;
        long ways = 0;
        for (int d = 0; d <= maxDigit; d++)
            ways += dfs(pos + 1, sum + d, tight && d == maxDigit);
        return memo[pos][sum][t] = ways;
    }
    static long count(long n, int k) {
```

```
        bound = Long.toString(n); limit = k;
        memo = new long[bound.length()][k + 1][2];
        for (long[][] a : memo) for (long[] b : a) Arrays.fill(b, -1);
        return dfs(0, 0, true);
    }
    public static void main(String[] args) {
        System.out.println(count(20, 2)); // 6: 0,1,2,10,11,20
    }
}
```

Complexity: Time O(L·K·10), space O(L·K).

Count LIS

File: src/dp/advanced/CountLIS.java

Package: dp.advanced | State / implementation rule: For each index store the best increasing-subsequence length ending there and how many ways achieve that length. O(n²) version is clear and interview-friendly; counts may overflow int on large inputs.

```
package dp.advanced;
public class LISCount {
    static int[] solve(int[] a) {
        int n = a.length;
        if (n == 0) return new int[]{0, 0};
        int[] len = new int[n], count = new int[n];
        java.util.Arrays.fill(len, 1);
        java.util.Arrays.fill(count, 1);
        int bestLen = 1, total = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (a[j] < a[i]) {
                    if (len[j] + 1 > len[i]) {
                        len[i] = len[j] + 1;
                        count[i] = count[j];
                    } else if (len[j] + 1 == len[i]) {
                        count[i] += count[j];
                    }
                }
            }
            bestLen = Math.max(bestLen, len[i]);
        }
        for (int i = 0; i < n; i++)
            if (len[i] == bestLen) total += count[i];
        return new int[]{bestLen, total};
    }
    public static void main(String[] args) {
        int[] ans = solve(new int[]{1,3,5,4,7});
        System.out.println("Length = " + ans[0] + ", count = " + ans[1]); // 4, 2
    }
}
```

Complexity: Time O(n²), space O(n).

Interview patterns and problem map

How to recognize the DP pattern

Clue in problem	Likely DP family	First state to try
Number of ways to reach / form target	1D, grid, counting knapsack	dp[position] or dp[sum]
Maximum/minimum value under a limit	0/1 or unbounded knapsack	dp[item][capacity]
Pick / not pick; adjacent picks forbidden	Linear / tree DP	dp[i][take/skip]
Two strings / subsequence / edits	String DP	dp[i][j]
Contiguous matching substring	Substring DP	ending-at-i,j state
Split array/string into segments	Partition / interval DP	dp[left][right] or dp[index]
Choose the last operation in a range	Interval DP	dp[left][right]
Buy/sell with restrictions	State-machine DP	dp[day][holding][transactions]
Tree with parent-child constraints	Tree DP	return states from children

Clue in problem	Likely DP family	First state to try
Visit subsets / assign people	Bitmask DP	dp[mask][last]
Count integers satisfying digit rules	Digit DP	pos, tight, started, extra state
DAG path max/min/count	DAG DP + topological order	dp[node]

High-priority interview problem checklist

• 1D: Fibonacci, Climbing Stairs, Min Cost Climbing Stairs, House Robber I/II, Decode Ways, Word Break, Maximum Product Subarray.

• Grid: Unique Paths I/II, Minimum Path Sum, Triangle, Minimum Falling Path Sum, Dungeon Game, Maximal Square, Cherry Pickup.

• Knapsack: 0/1 Knapsack, Partition Equal Subset Sum, Subset Sum, Target Sum, Last Stone Weight II, Coin Change I/II, Combination Sum IV, Ones and Zeroes.

• Sequence: LIS, Number of LIS, Russian Doll Envelopes, LCS, Longest Common Substring, Edit Distance, Delete Operation for Two Strings, Distinct Subsequences, Interleaving String, Shortest Common Supersequence, Palindromic Substrings, LPS.

• Interval: Matrix Chain Multiplication, Burst Balloons, Minimum Cost to Cut a Stick, Palindrome Partitioning II, Strange Printer, Remove Boxes.

• Stocks: Best Time to Buy and Sell Stock I/II/III/IV, Cooldown, Transaction Fee.

• Trees/graphs: House Robber III, Binary Tree Maximum Path Sum (tree DP), longest path in DAG, count paths in DAG.

• Bitmask: Traveling Salesperson, Assignment Problem, Smallest Sufficient Team, Shortest Path Visiting All Nodes.

• Digit DP: count numbers with digit sum, digit restrictions, no repeated adjacent digits, count numbers in range [L,R] using F(R)-F(L-1).

Important DP algorithms / optimizations to learn after core DP

• Topological-order DP on DAGs: compute indegrees, Kahn's queue, then relax outgoing edges in topological order.

• Space optimization: remove dimensions only when dependency order proves old values are no longer needed.

• Monotonic queue optimization: sliding-window min/max transitions.

• Divide-and-conquer DP optimization: when the optimal split point is monotone; typical $O(kn^2)$ to $O(kn \log n)$.

• Knuth optimization: interval DP under required quadrangle/monotonicity conditions; typical $O(n^3)$ to $O(n^2)$.

• Convex Hull Trick / Li Chao Tree: optimize transitions of the form min/max(m*x+b).

• SOS DP: subset sums over all submasks/supermasks, often O(n·2^n).

• Profile DP: grid tiling/domino placement with bitmask states.

Complexity reference

• O(n): linear DP with constant work per state.

• O(nW): knapsack with n items and capacity W.

• O(mn): two-dimensional string/grid DP.

• $O(n^2)$: classical LIS and many interval precomputations.

• $O(n^3)$: matrix-chain / burst-balloons interval DP.

• O(n·2^n) or $O(n^2 \cdot 2^n)$: subset/bitmask DP; n is usually small.

• Digit DP: commonly O(number of digits × auxiliary states × 10).

Final correctness checklist before submitting in an interview

- Can I explain the meaning of every state without hand-waving?
- Have I included empty input / smallest valid input?
- Are impossible states represented safely?
- Does the loop order match 0/1 vs unbounded reuse?
- Am I counting combinations or permutations as requested?
- Are negative values allowed? If yes, does the initialization still work?
- Can the answer exceed int? Should I use long or modulo?
- Have I proved why the space compression preserves dependencies?
- Can I dry-run a tiny example and one edge case?

Scope note: This volume gives full runnable code for the core representative algorithms above and a broad interview checklist. The advanced optimization families and several named interview problems are explained as patterns/checklist items, not each supplied with a separate full implementation in this volume. They should be expanded into a dedicated advanced-DP code volume rather than falsely described as already solved here.